

**INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA**
SUL-RIO-GRANDENSE
Campus Passo Fundo

Bacharelado em Ciência da Computação
Arquitetura de Computadores I

Conteúdo 5:
Sistema de memória:
Memória Principal (Ram/Rom) e Cache

Prof. Petry, Carlos A

AGENDA

- Sistema de memória
 - Hierarquia de memórias
 - Características das memórias
- Memória principal (DRAM)
 - Memória primária
 - Organização
 - Tecnologias
 - RAM x CPU
 - Temporização
- Memória Cache (SRAM)
 - Mapeamento de memória
 - Políticas de escrita de cache
 - Tipos de cache
 - Cache de processador
 - Função
 - Virtual
- Memória de troca (Swap)
 - Tradução de endereços
 - Tabela páginas
 - Política de substituição
 - Fragmentação

The slide features decorative circuit-like lines in the corners. The top-left and top-right corners have dark green lines, while the bottom-left and bottom-right corners have light green lines. A thin brown horizontal line is positioned near the top center of the slide.

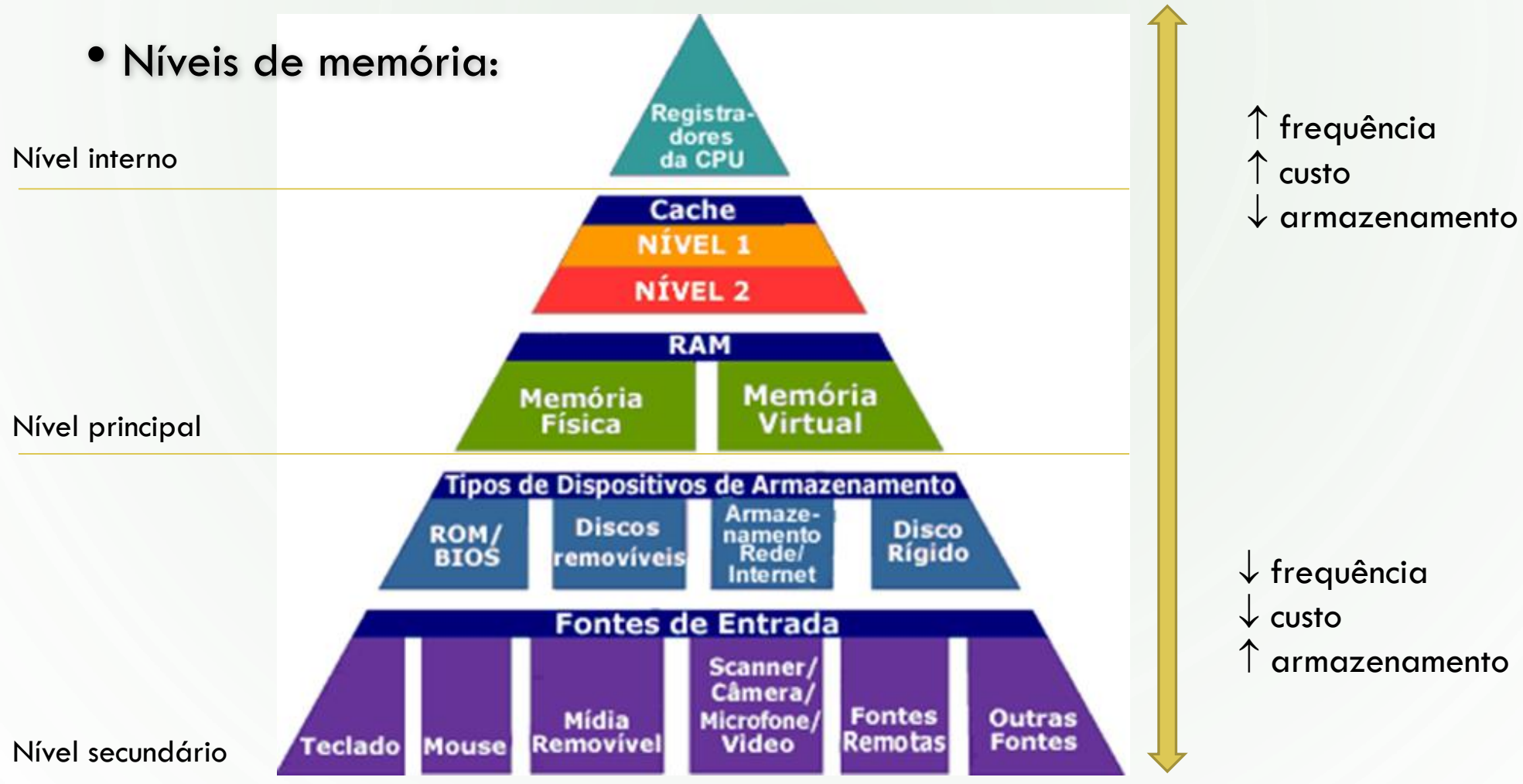
SISTEMA DE MEMÓRIA

HIERARQUIA DE MEMÓRIAS

- Necessitamos diferentes tipos de memória :
 - Devido a características tecnológicas
 - Para obter melhor desempenho ao acessar dados
- Existem diferentes propósitos ao acessar a memória :
 - Processamento → velocidade de transferência e acesso direto
 - Armazenamento → alta capacidade e persistência
- Hierarquia de memória implica diferentes níveis de acesso
 - Os diversos níveis são interligados estabelecendo a hierarquia de memória

NÍVEIS DA HIERARQUIA

- Níveis de memória:



(Fonte: <http://informatica.hsw.uol.com.br/memoria-do-computador1.htm>)

IMPLICAÇÕES DA HIERARQUIA

- Comparativo quanto à posição no nível hierárquico

↑

Mais próximo

Menor

Menor

Maior

↓

Mais distante

Maior

Maior

Menor

(do processador)

a capacidade de armazenamento

o tempo de acesso

o custo de armazenamento

- Vantagens da hierarquia

- Trazer os **dados** o mais próximo do processamento possível, segundo algum critério
- Possibilitar que requisições de acesso à memória consumam o menor tempo possível

- Acesso aos dados nos diversos níveis da hierarquia
 - Os dados podem estar em qualquer nível da hierarquia
 - Porém sempre transitam do:
 - nível mais distante em direção para o mais próximo do processador
- Transferência de dados entre os níveis da hierarquia
 - Ocorre sob demanda do processador (programas em execução)
 - O processador carrega dados e instruções da hierarquia de memória
 - No 1º acesso os dados estarão possivelmente na memória secundária (?)
 - Os dados são sempre carregados do nível mais próximo possível da CPU
 - Caso o dado não seja encontrado (falha), o próximo nível (mais distante) é acessado (sucessivamente)
 - Ao serem encontrados, blocos de dados serão copiados para cada nível da hierarquia até chegarem ao processador



CARACTERÍSTICAS DAS MEMÓRIAS

- **Volatilidade**

- Volátil → **dados** são perdidos na falta de energia elétrica
- Não-volátil → **dados** são preservados na falta de energia elétrica

- **Capacidade**

- Quantidade de dados possíveis de serem **armazenados** na memória

- **Tempo de acesso**

- Tempo necessário para realizar uma **operação em memória** (leitura | escrita), em geral medido em nanosegundos (ns)

- **Tecnologia de fabricação de memórias**

- Semicondutor → construídas em geral com base no silício
- Magnético → construídas com material magnetizável, em geral (Fe_2O_3 - óxido de ferro/óxido férrico)
- Óptico → construída com materiais reflexivos operado com laser, em geral com materiais cristalino+amorfo

- **Temporariedade**

- Tempo que a informação permanece nos diferentes tipos de memória
(persistente ou transitória)

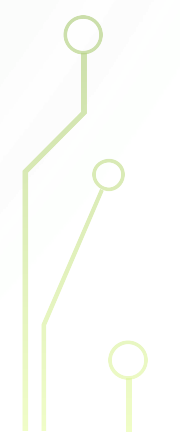
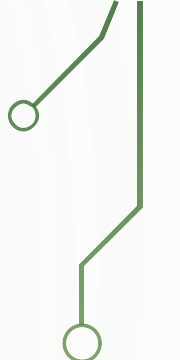
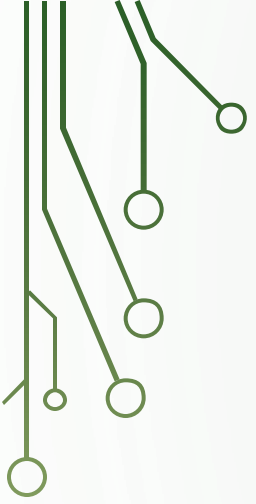
- **Memórias de semicondutor: RAM, ROM, Flash**

- **RAM:** memória de leitura e escrita

- **ROM:** memória apenas de leitura

- **Flash:** memória de leitura e escrita de armazenamento não volátil

Tipo de memória	Categoria	Apagamento	Mecanismo de escrita	Volatilidade
Memória de acesso aleatório (RAM)	Memória de leitura-escrita	Eletricamente, em nível de byte	Eletricamente	Volátil
Memória somente de leitura (ROM)	Memória somente de leitura	Não é possível	Máscaras	Não volátil
ROM programável (PROM, do inglês <i>programmable ROM</i>)			Eletricamente	
PROM apagável (EPROM, do inglês <i>erasable PROM</i>)	Luz UV, nível de chip			
PROM eletricamente apagável (EEPROM, do inglês <i>electrically erasable PROM</i>)	Eletricamente, nível de byte			
Memória flash	Eletricamente, nível de bloco			

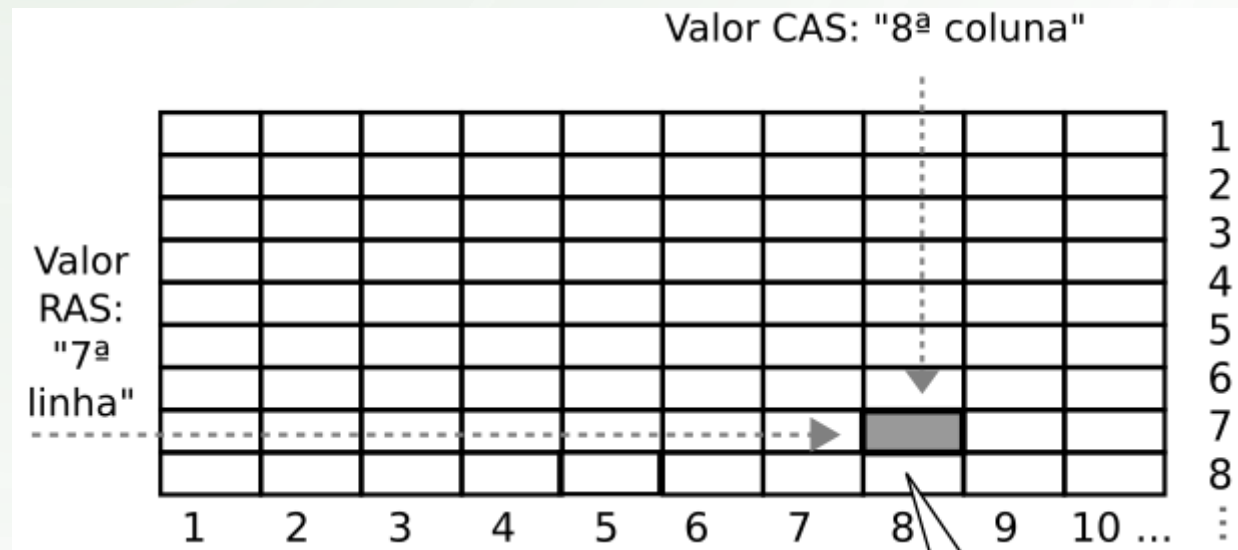
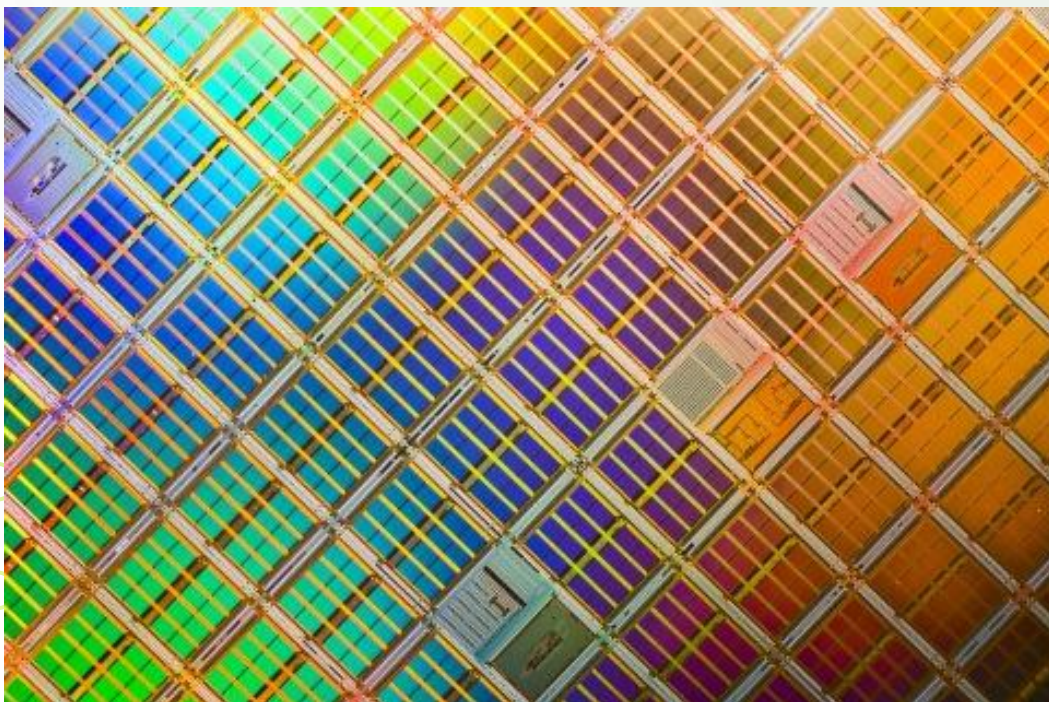


MEMÓRIA PRINCIPAL: RAM

MEMÓRIA RAM

- Memória acessada pelo processador para obter dados e instruções para o processamento
- É organizada em formato de **células** de memória endereçáveis
- Cada célula contém uma quantidade fixa de bytes (word), dependente da arquitetura, como: 32 ou 64 bytes
- Endereços: iniciam em 0 (zero) e seguem até o tamanho da memória (-1)
- Acesso aos endereços ocorre de forma alinhada com a palavra (word): 0, 4, 8, ... No caso de 4bytes (32bits)
- A granularidade ocorre em nível de byte, endereçável por Byte ou Word
 - Locais de memória costumam ser acessados por meio de variáveis
 - Variáveis são 'sinônimos' para endereços de memória, em programas
- Cada célula é acessada de forma integral (como um todo)

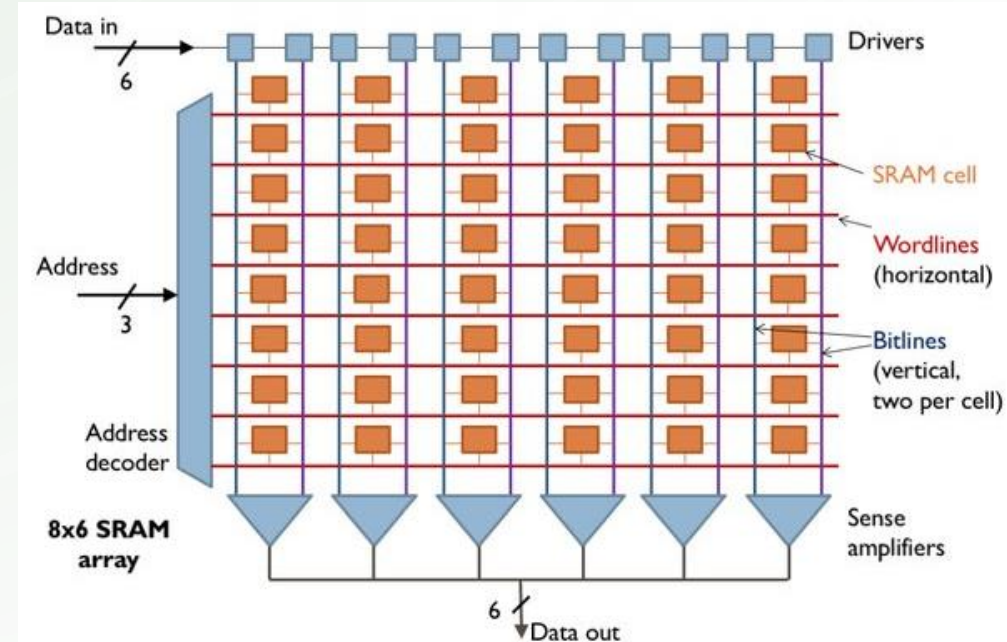
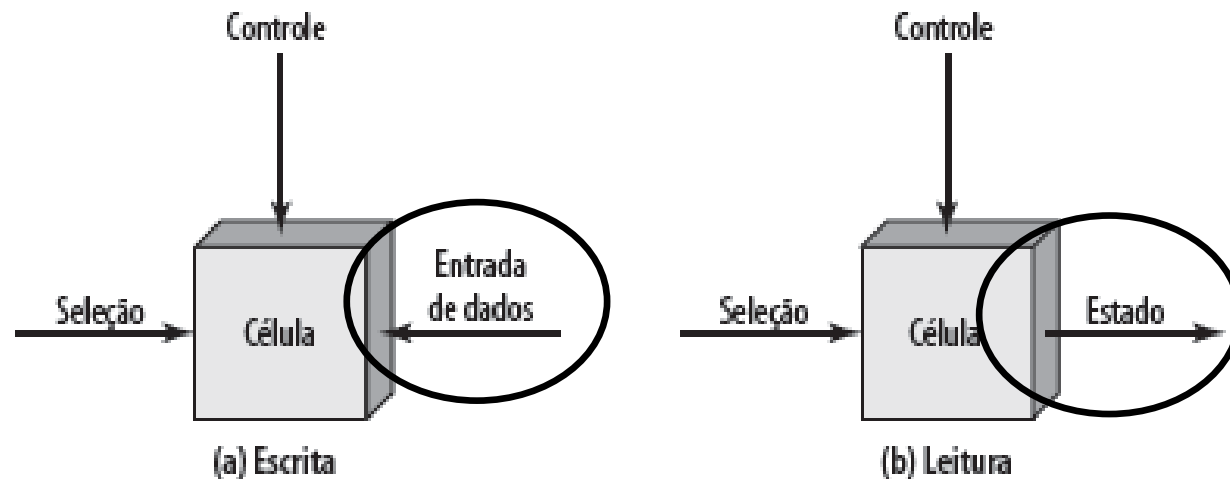
- RAM: memória de acesso aleatório ou *Random Access Memory*
 - Aleatório: significa possibilidade de acesso à qualquer endereço (posição alinhada) a qualquer momento
 - A estrutura (física) de um chip-RAM é formada por células idênticas organizadas como linhas e colunas (matriz)



- Componentes de acesso à memória principal

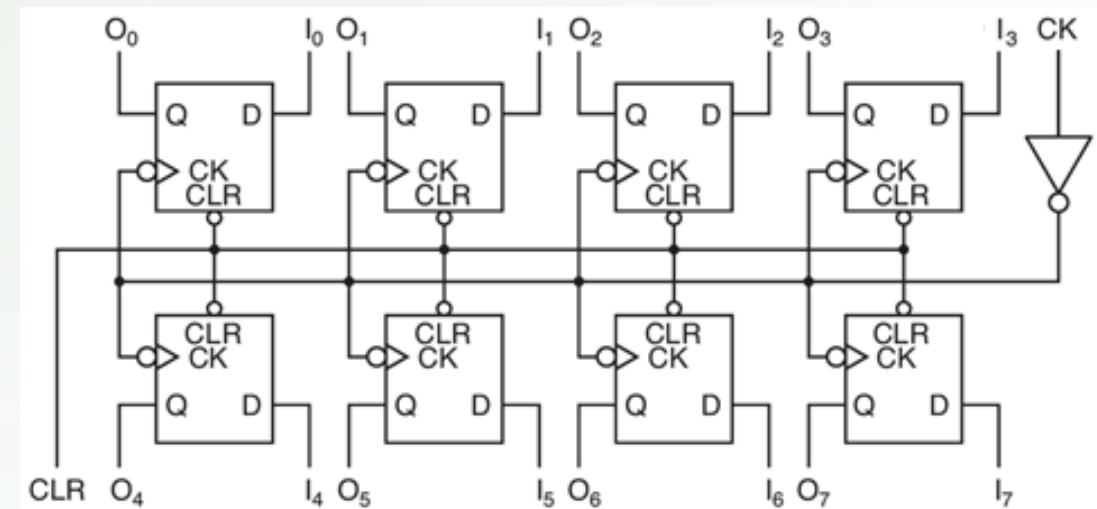
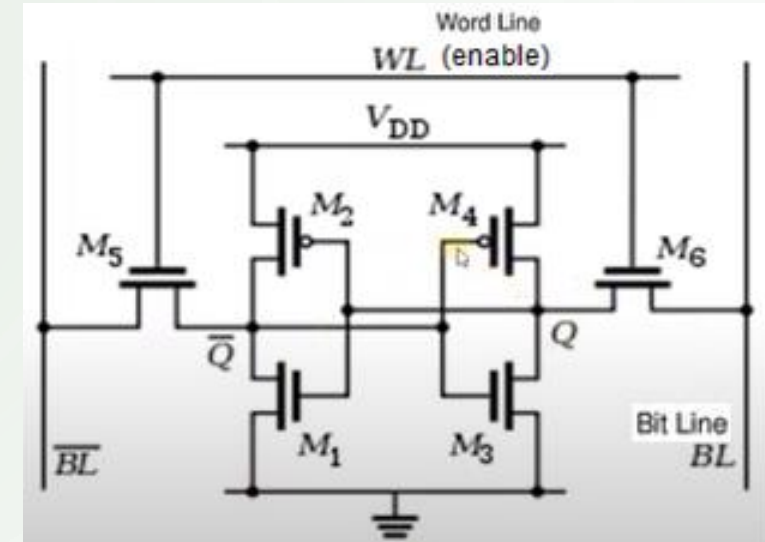
- **Processador:** requisita o acesso à memória
- **Controlador de memória:** responsável por gerenciar o acesso à memória
- **Barramento:** caminho que conecta os dispositivos, como: processador e memória RAM

- Operações: seleção (endereçamento), leitura e escrita



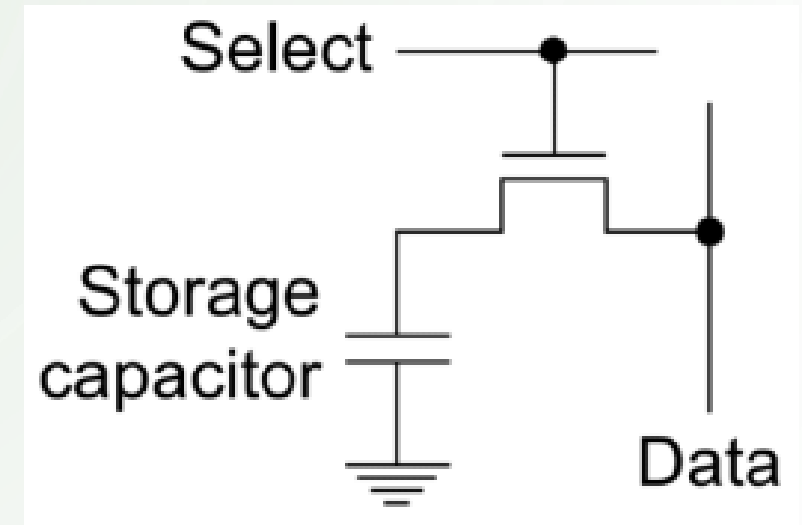
• Memórias RAM estáticas

- SRAM → Static RAM
- Dados armazenados via carga elétrica
- Para armazenar um bit de dado
 - Basta inserir o valor na entrada ou ler da saída, em geral usam Flip-Flop
- Mantém o dado armazenado enquanto energizada



• Memórias RAM dinâmicas

- DRAM → Dynamic RAM
- Dados armazenados via carga capacitiva
- Armazenam um bit de dado
 - o **transistor** controla a passagem da energia
 - o **capacitor** armazena a energia, por certo tempo
- Necessidade de recarga periódica do capacitor:
refresh



- Barramentos de acesso à memória

- O barramento compreende o caminho para acesso à memória
- Existem três tipos de barramentos:
 - **Dados:** por onde transitam os dados (lidos ou escritos)
 - **Endereço:** por onde é seccionada o endereço (posição) do dado
 - **Controle:** por onde transitam os sinais de controle (sincronismo) das operações realizadas (W | R), como clock, enable, write

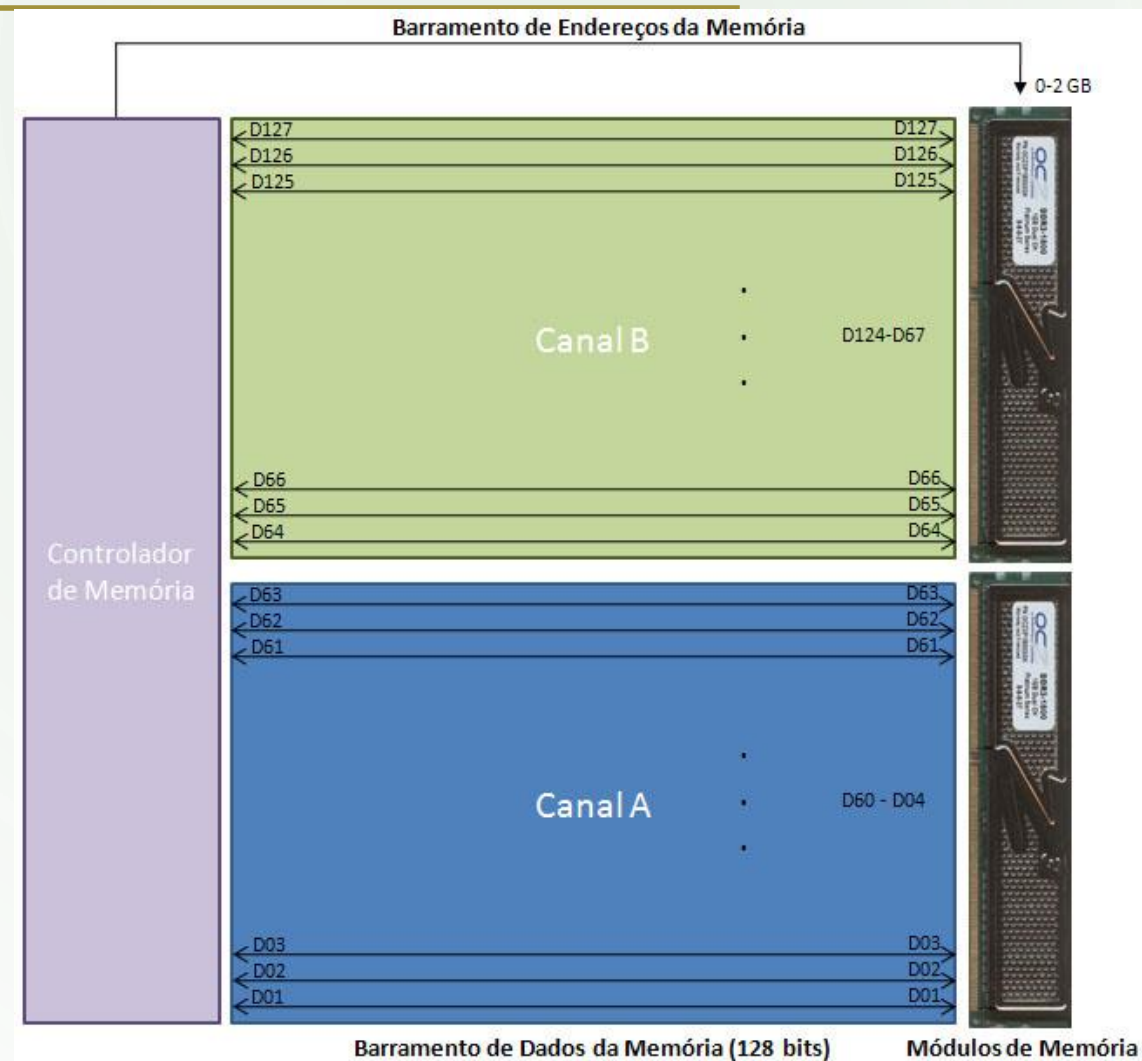
MECANISMOS CPU-RAM

- Processadores são mais rápidos do que a memória RAM
- Precisamos de mecanismos de sincronismo/control de acesso
 - Usar de memória cache: memórias RAM mais rápidas
 - Aumentar da largura do barramento de dados: 32b → 64b → 128b
 - Multiplicar os canais de acesso aos módulos RAM: multi-channel

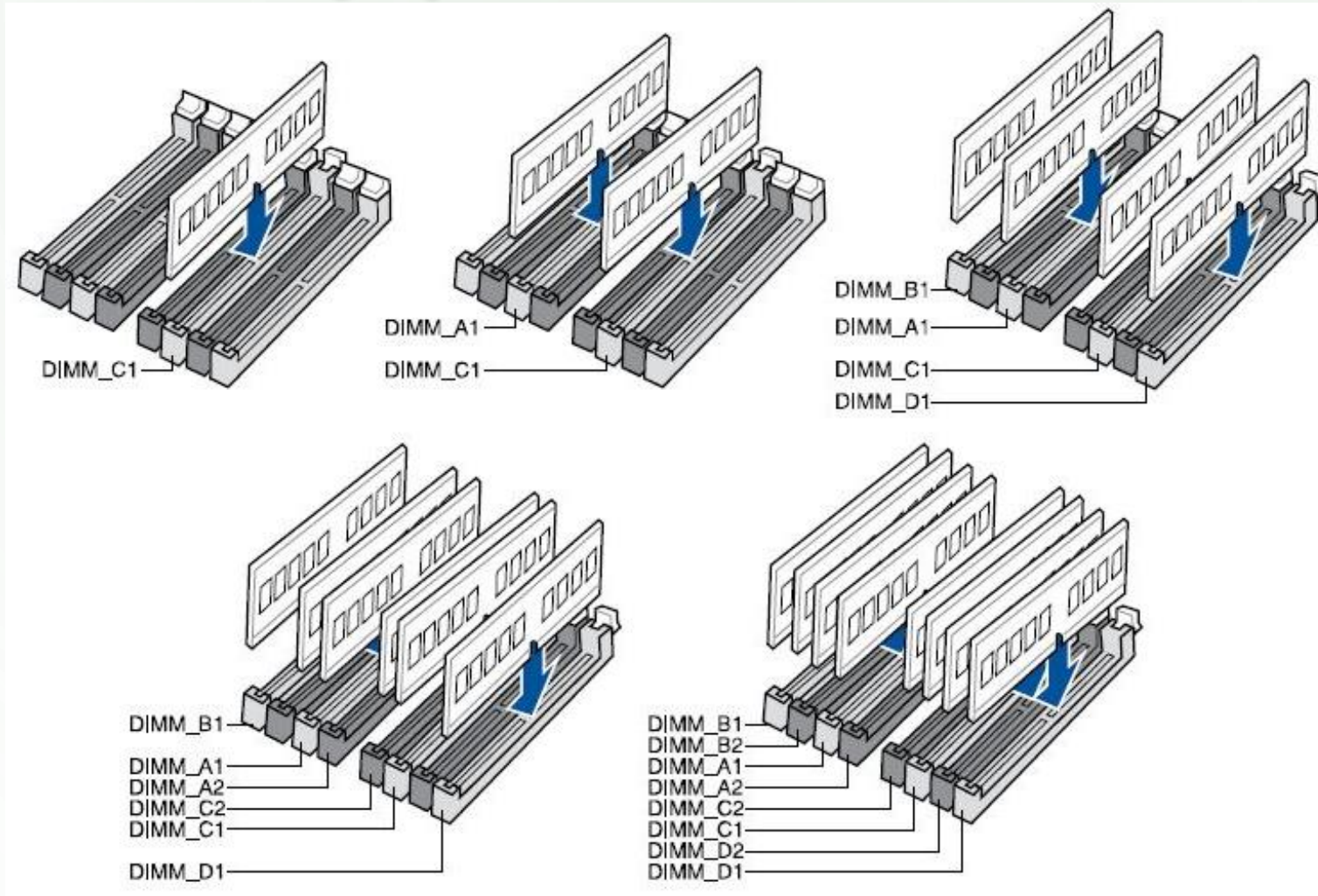
- **Multi-channel:**

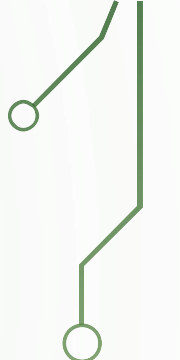
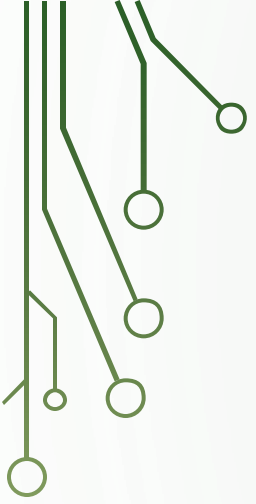
- Possibilidade de **acesso simultâneo** a múltiplos módulos de memória
- Acesso à memória via canais:
 - Dual → 128 bits (2x 64)
 - Triple → 192 bits (3x 64)
 - Quad → 256 bits (4x 64)
- Aumenta a quantidade de dados transferidos para o processador a cada ciclo de clock

MULTI-CHANNEL: DUAL



- Multi-channel: configurações

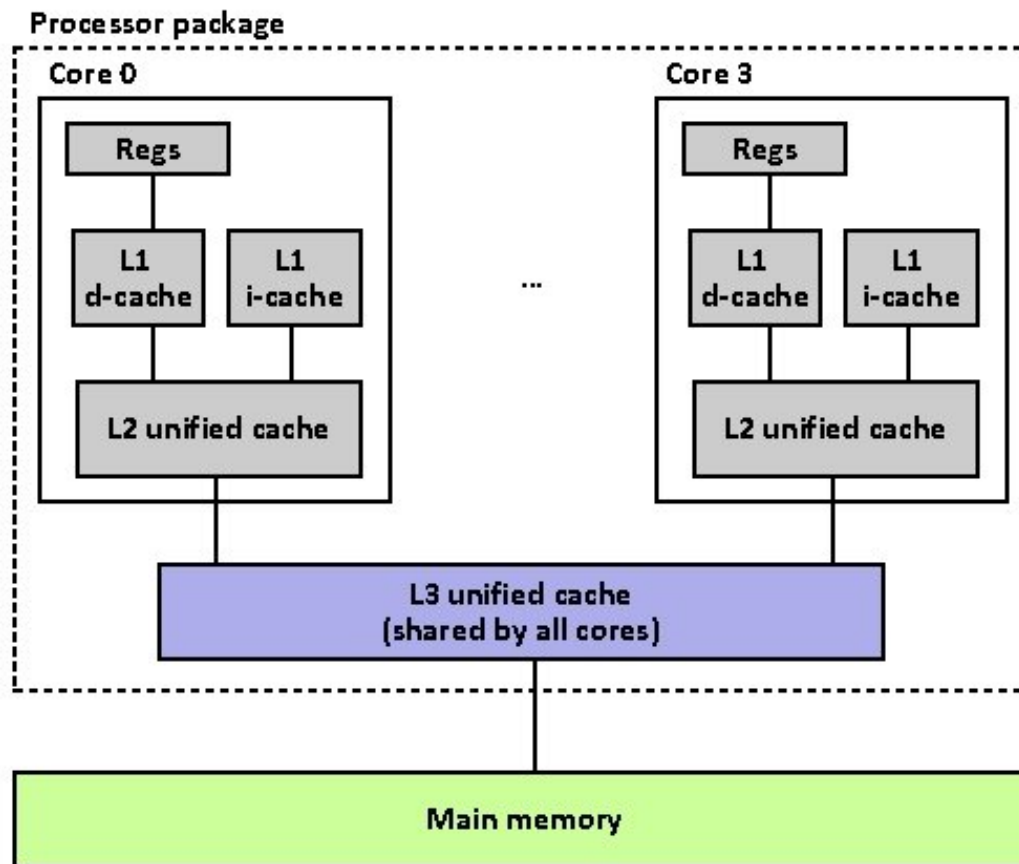




MEMÓRIA CACHE (SRAM)

- Níveis de memória Cache: o exemplo do i7

Intel Core i7 Cache Hierarchy



- Acessando dados com sistemas com cache:

- Quando o processador precisa acessar dados inicialmente **verificada se está na cache**
- Se encontrar, utiliza o dado em cache
- Se não encontrar, segue a busca no nível seguinte (mais distante do processador)

- Cache hit (acerto)

- Sendo o dado acessado na memória cache localizado, ocorre um **cache hit**
- **HIT**: evita a busca de dados nos níveis seguintes da hierarquia de memória
- **Taxa de acerto**: o percentual de acessos que resultam em sucesso na cache:
cache hits → hit rate

- Cache miss (erro/falta)

- Não sendo localizado o dado na memória cache, ocorre um **cache miss**
- **MISS**: obriga buscar os dados nos níveis seguintes da hierarquia de memória e copia-los para os níveis anteriores (mais próximos do processador)
- **Taxa de erro**: o percentual de acessos que resultam em falha na cache:
cache miss → miss rate

PRINCÍPIO DA LOCALIDADE

- Localidade espacial
 - Sendo um posição de memória acessada em um dado momento, há grande probabilidade de um local próximo ser acessado em breve
 - Assim, é comum dimensionar blocos de memórias cache para um tamanho que otimize acessos subsequentes
- Localidade temporal
 - Sendo um posição de memória acessada em um dado momento, há grande probabilidade que o mesmo local seja acessado em breve
 - Assim, é comum manter dados em memórias cache para otimizar acessos subsequentes ao mesmo dado

▶ Linha de cache

- Memórias RAM e Cache são organizadas em blocos de tamanho fixo
- Blocos em Cache são chamados de **linhas de cache**, em geral de 4 a 64 bytes
- Ao ocorrer um **cache miss** é transferido da RAM: uma linha completa da cache
- A transferência em bloco potencializa o princípio da localidade

• Conflito em cache

- Ao transferir blocos para a cache uma ou mais linhas podem estar ocupadas com outros dados, ocorrendo um conflito de cache → cache conflict
- Ao ocorrer o conflito é necessário liberar a linha na memória cache: substituir os dados
- Política de troca: mecanismo para definir qual linha de cache será substituída ao inserir novos dados (replacement policy)
- Existem várias políticas de troca, as duas mais comuns são:
 - LRU** (Least Recently Used): remove a linha recentemente menos usada
 - LFU** (Least Frequently Used): remove a linha frequentemente menos usada

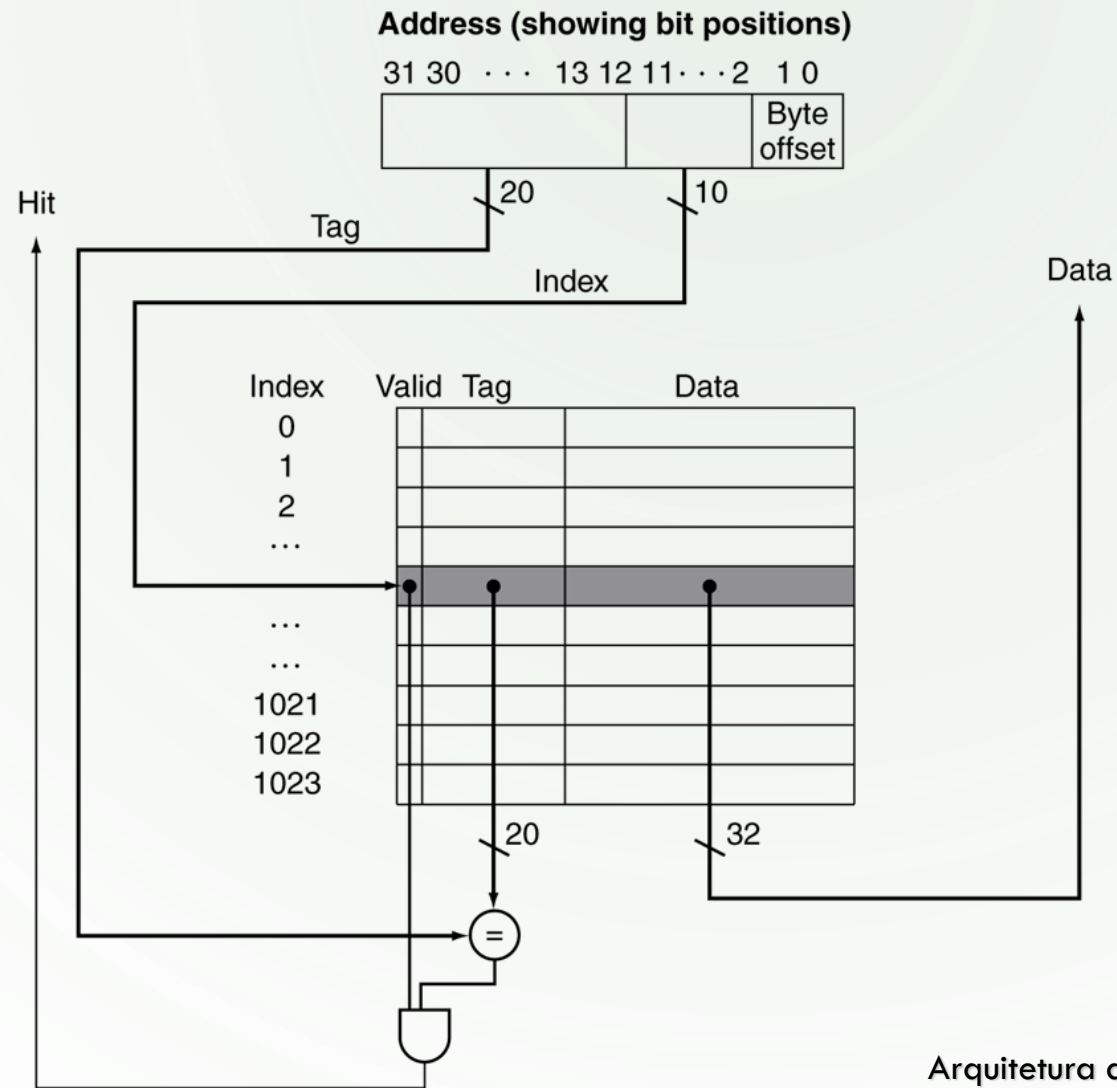
MAPEAMENTO DE MEMÓRIA

- A memória cache é menor do que a RAM
- Assim, não é possível alocar toda a RAM na Cache
- Se a cache fosse a metade da RAM, dois blocos da RAM concorreriam pela mesma linha da cache
- Para resolver esta situação usamos métodos ou Funções de Mapeamento
 - Define qual bloco de memória RAM ocupa uma linha na cache

FUNÇÕES DE MAPEAMENTO

- Mapeamento direto (Direct Mapping)
 - Representa a técnica mais simples de mapeamento
 - Cada bloco de memória RAM é mapeado para uma linha de cache pré-definida
 - Vantagem:
 - Bloco da RAM é rapidamente (direto) localizado na cache
 - Desvantagem:
 - Mais de um bloco de RAM pode ser mapeado para a mesma linha da cache → conflito
 - Acesso à blocos de memória tendem a gerar trocas frequentes de dados nas linhas da cache
- Campos de controle e dados
 - Valid Bit : indica se a linha de cache possui um dado válido
 - Tag/Index: conjunto de bits equivalente a uma porção MSB do endereço do dado na RAM, usado para validar se o bloco da RAM está na Cache
 - Byte Offset: indexa o byte (dado da RAM) existente na linha da cache

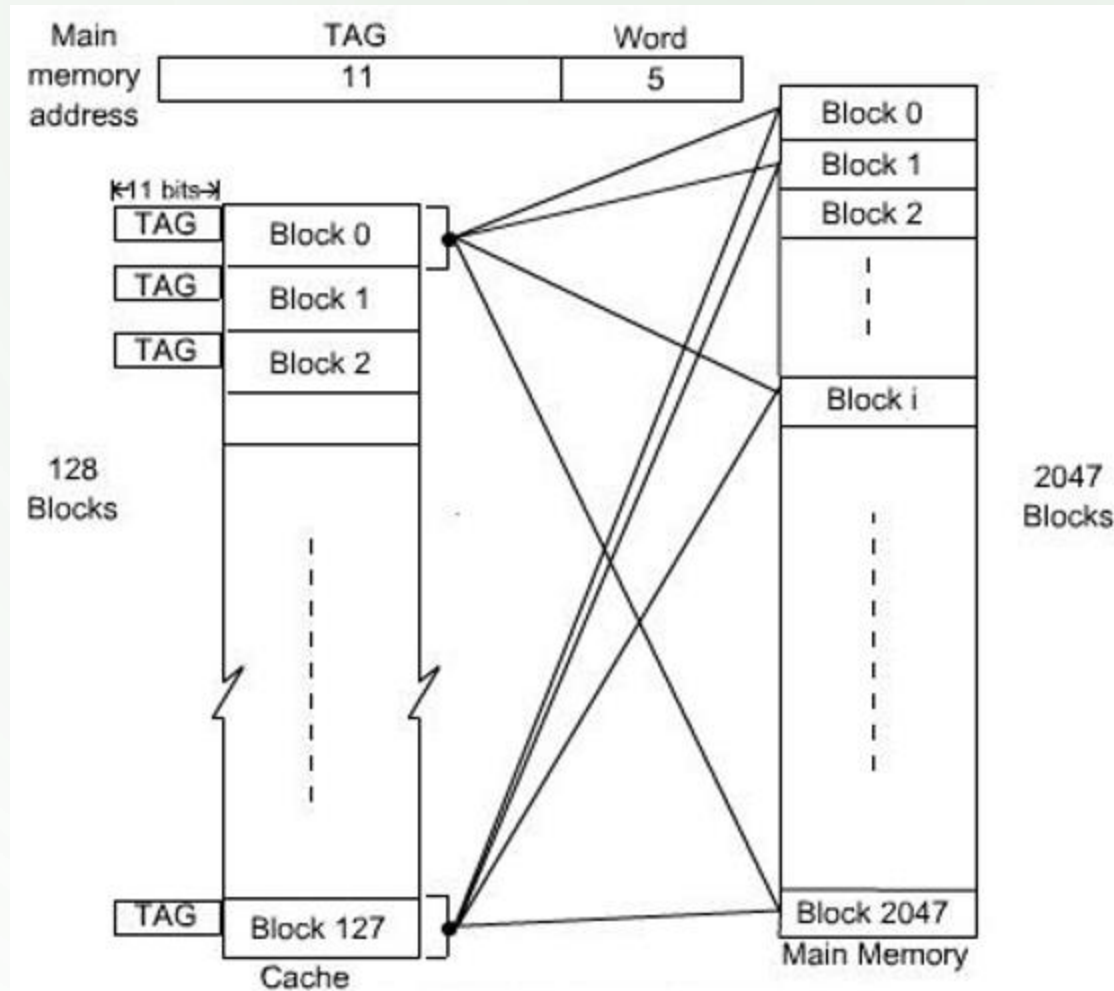
- Mecanismo de funcionamento:



FUNÇÕES DE MAPEAMENTO

- Mapeamento completamente associativo (Fully Associative Mapping)
 - Qualquer bloco de memória RAM pode ser mapeado para qualquer linha da cache, não há locais pré-definidos
 - Vantagem:
 - Reduz ao máximo problemas de conflitos de blocos da RAM com linhas da cache
 - Desvantagem
 - Localizar dados nas linhas da cache requer uma busca sequencial → desempenho
- Campos de controle e dados
 - Valid Bit : indica se a linha de cache possui um dado válido
 - Tag: conjunto de bits equivalente a uma porção MSB do endereço do dado na RAM, usado para validar se o bloco da RAM está na Cache
 - Byte Offset: indexa o byte (dado da RAM) existente na linha da cache

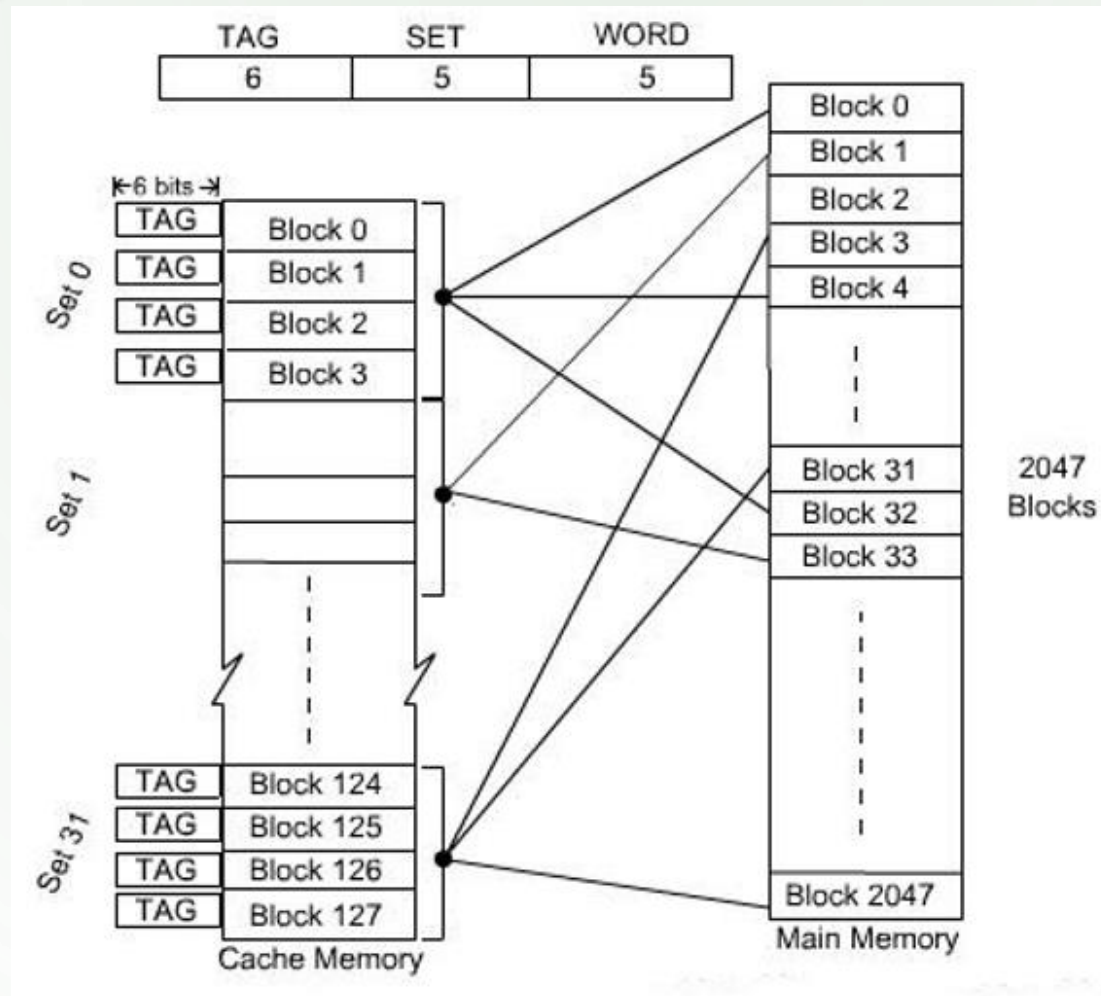
- Mecanismo de funcionamento:



FUNÇÕES DE MAPEAMENTO

- Mapeamento associativo por conjuntos (Set Associative Mapping)
 - Cada bloco de memória RAM é mapeado para um conjunto (set) de linhas de cache pré-definidos
 - Cada bloco da RAM pode ser inserido em qualquer linha do conjunto associado
 - Vantagem:
 - Une a flexibilidade do Fully Associative com o desempenho do Direct Mapping
 - Conflitos de mapeamento são menores que o Direct Mapping e os dados são localizados mais rapidamente que no Fully Associative
 - Acesso à blocos de memória tendem a gerar menor trocas de dados nas linhas da cache
- Campos de controle e dados
 - Valid Bit : indica se a linha de cache possui um dado válido
 - Tag/Index: conjunto de bits equivalente a uma porção MSB do endereço do dado na RAM, usado para validar se o bloco da RAM está na Cache
 - Byte Offset: indexa o byte (dado da RAM) existente na linha da cache

- Esquema de funcionamento



Políticas de escrita da Cache

- Ao escrever um dado em cache a cópia correspondente na RAM torna-se desatualizado
- Em algum momento o dado precisa ser atualizado na RAM
- Política de Escrita (Write Policy):
 - Define em que momento a RAM será atualizada com base no conteúdo da Cache

- Política Write-through (“escrita através”):

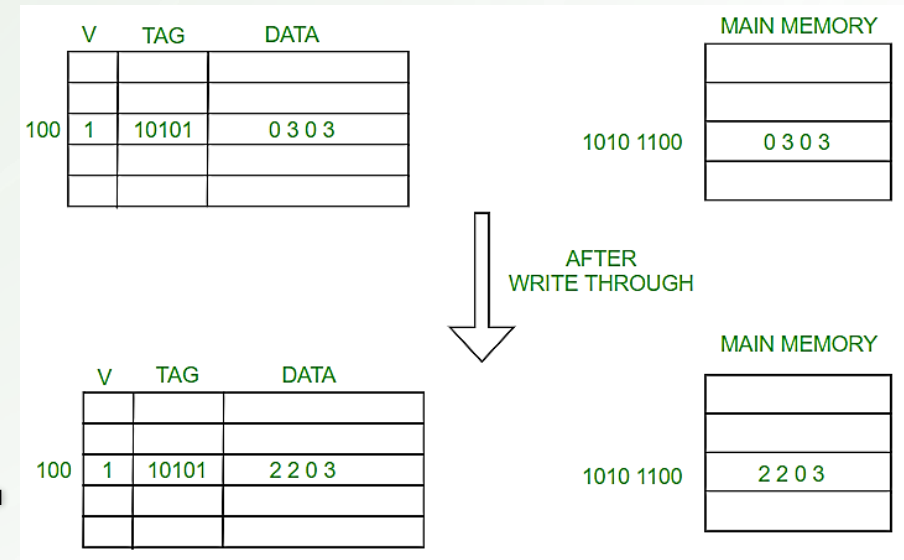
- Toda vez que ocorre uma escrita na Cache o mesmo dado será também escrito na RAM (escrita sincronizada)

- Vantagens

- A hierarquia de memória permanece sempre atualizada, não ocorre inconsistência
- Facilita a troca do conteúdo da Cache

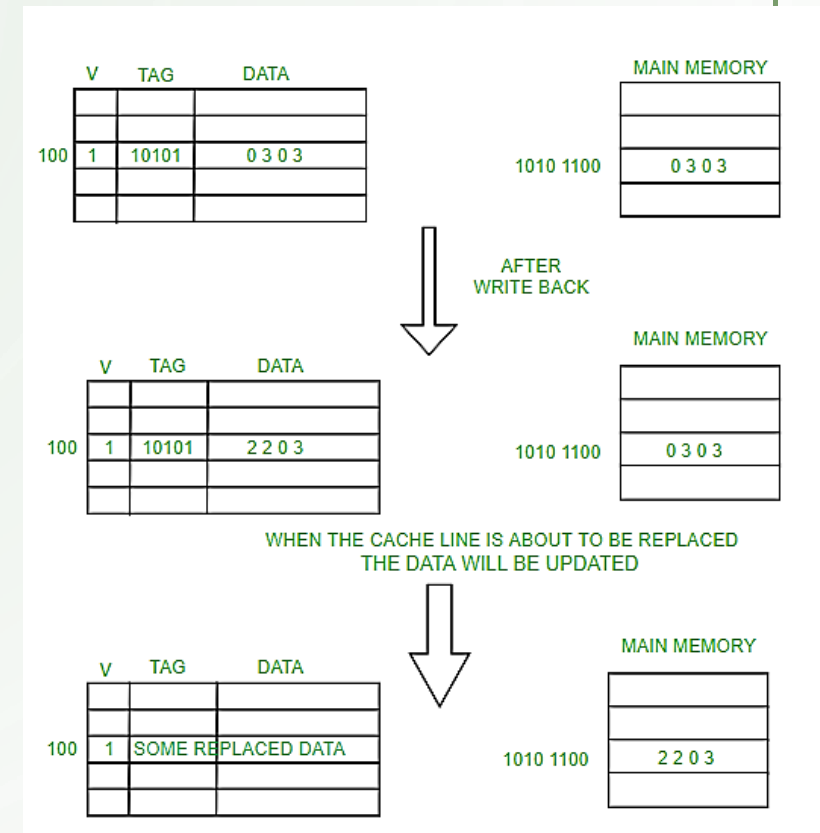
- Desvantagens

- A escrita sincronizada torna o sistema de memória mais lento
- É necessário aguardar a conclusão da escrita do dado em toda a hierarquia



- Política Write-back (escrever de volta):

- Escritas na Cache não atualizam o mesmo dado na RAM (escrita assíncrona)
- A sincronização dos dados na Cache será realizada em momento posterior ou quando a linha de Cache for substituída na Cache
- Vantagens
 - Permite atualizar apenas a última versão do dados na ocorrência de múltiplas escritas na Cache
 - Menor número de sincronização melhora o desempenho do sistema de memória
- Desvantagem
 - Os dados podem ficar inconsistentes, necessitando mecanismos de coerência de dados
 - Novas escritas na linha da Cache precisam esperar pelo sincronismo



Coerência de Cache

- Coerência de Cache
 - Os dados disponíveis nos locais de armazenamento originais **podem ser modificados por outros** componentes do sistema, além do Cache
 - Risco: a cópia existente no cache pode se tornar inválida
 - Quando se atualiza dados em um nível da cache, cópias do dado em outros níveis (Cache/Ram) tornam-se inválidos
- Protocolos de coerência de cache
 - Responsáveis por manter os dados consistentes entre os níveis de Cache e a Ram

Tipos de Cache

- Cache unificada – Arquitetura Von Neumann
 - A mesma área da Cache pode **armazenar instruções ou dados**
- Cache dividida – Arquitetura Harvard
 - A cache é dividida em duas áreas distintas
 - **Armazenamento de dados**
 - **Armazenamento de instruções**
 - Permite ao processador acessar instruções (na cache de instruções) e dados (na cache de dados) **simultaneamente**
 - Em geral os programas não alteram suas instruções, facilitando descartar instruções não mais usadas
 - problema quando um programa se automodifica (alterações devem ser escritas na área de dados e na memória principal);
 - Geralmente instruções ocupam menos armazenamento, possibilitando dedicar mais espaço para dados

Níveis de Cache

- Atualmente sistemas computacionais utilizam Cache multinível
 - Cache L1 (level 1), cache L2 (level 2) e cache L3 (level 3);
- Cache L1
 - Mais próxima do núcleo do processador
 - Opera a mesma velocidade do processador e possui baixa latência
 - Novas tecnologias de processadores exigem mudanças na tecnologia da Cache L1
 - Costumam ser implementadas como cache dividida
- Demais níveis de Cache (L2 ou superior)
 - Costumam ser implementadas como cache unificada
 - Níveis hierárquicos superiores (+ distantes da CPU) apresenta maior capacidade e maior tempo de acesso

Níveis de Cache

- Processador: mecanismo de acesso à memória
 - Quando o processador precisa ler dados na memória principal (Ram), o controlador de cache transfere blocos de dados da RAM para os níveis de cache
 - O acesso inicia sempre no **Cache L1**
 - Caso o dado seja encontrado é diretamente enviado o processador
 - Caso o dado não seja encontrado, o próximo nível de Cache é acessado, e assim sucessivamente até chegar à RAM e transferido (em bloco) para os níveis inferiores de Cache (mais próximos aos processador)
 - Para dados não encontrados nos níveis de cache, serão necessários vários ciclos de clock para obter os dados (bloco) da memória RAM

Hierarquia de memória

- Dimensões* dos vários níveis de memória (*não proporcional)

Memória RAM



Cache L3



Cache L2



Cache L1



Processador



Níveis de Cache

- Tempo de acesso típicos da hierarquia de memória:

- Cache L1: cerca de 3 a 4 ciclos
- Cache L2: cerca de 10 a 15 ciclos
- Cache L3: cerca de 30 a 50 ciclos
- Memória RAM: maior que 150 ciclos

Level	Memory Module	Size	Access Clock Cycles
1	L1 Cache	32KB per core	4
2	L2 Cache	256-512KB per core	11
3	L3 Cache	1-3 MB per core	40
4	DRAM to a blade	128GB	$O(200)$
5	DRAM to other blades	128GB and more	$O(1500)$

On-chip memory (SRAM)	Off-chip memory (DRAM)	Solid State Disk (Flash Memory)	Secondary Storage (HDD)
Latency: (Cycles)			
1~30	100~300	25000~2000000	>5000000

¹ Fonte: Scalable 3D hybrid parallel Delaunay image-to-mesh conversion algorithm for distributed shared memory architectures. (2016)

² Fonte: <https://cseweb.ucsd.edu/classes/wi17/cse237A-a/handouts/03.mem.pdf>

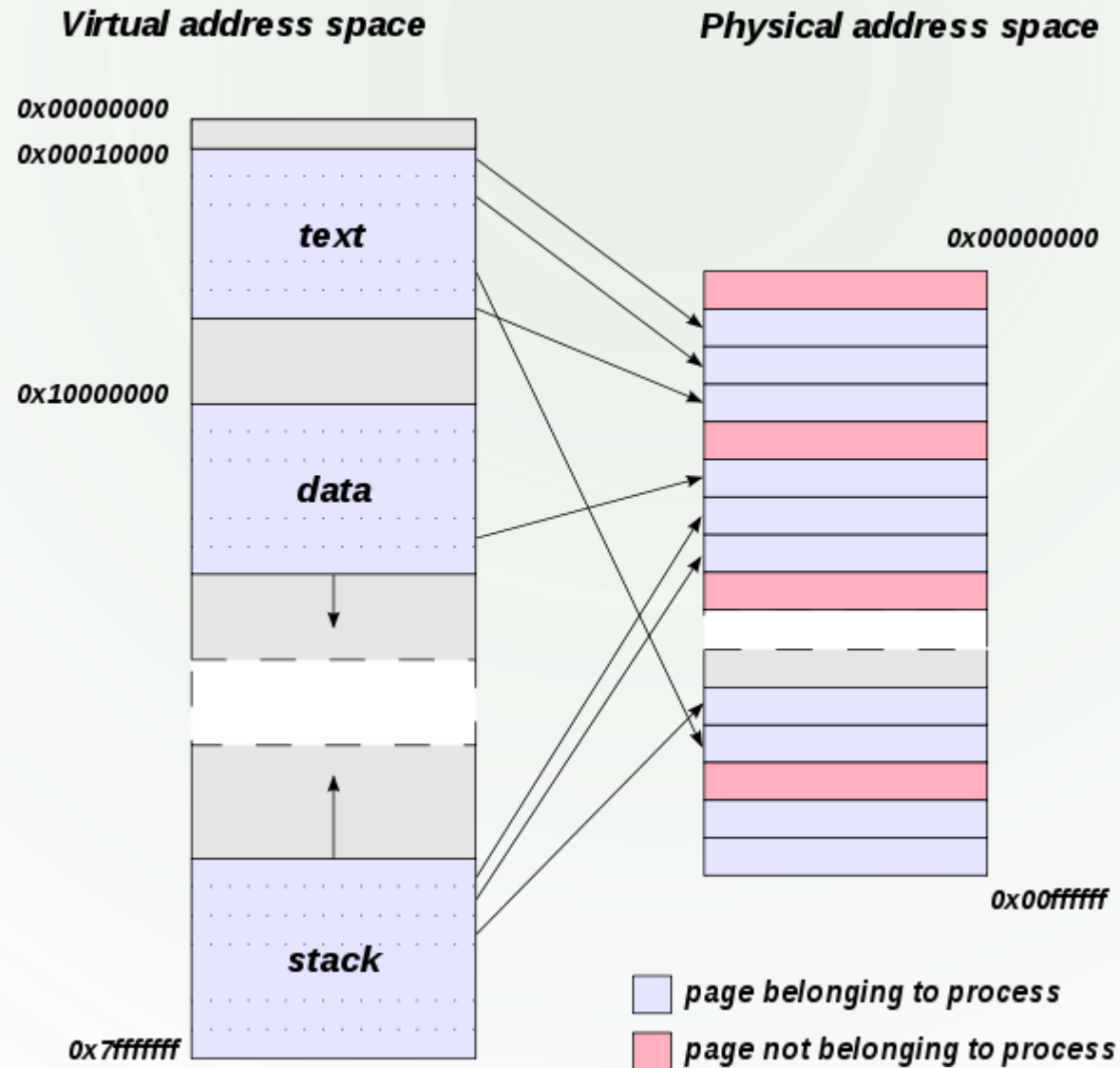
FUNÇÃO

- Cache em micros antigos:
 - era limitada a armazenar **cópias das últimas informações** acessadas pelo processador e descartava as informações mais antigas ou menos acessadas;
 - eram unificadas;
- Cache em micros atuais:
 - Conta com o mecanismo de prefetch
 - Monitora o fluxo de instruções e **carregam antecipadamente dados** que poderão ser necessários em ciclos seguintes
 - Aceleraram operações de escrita, permitindo que o **processador grave os dados diretamente na cache**, ficando o controlador de cache responsável por gravar os dados na memória RAM, posteriormente

ENDEREÇOS FÍSICOS E VIRTUAIS

- Endereços físicos
 - Local onde os dados (e instruções) são armazenados fisicamente na memória principal
 - Necessário para execução e acessos à memória Ram
 - São manipulados (reconhecido) pelo sistema operacional (system space)
 - São obtidos pelo processo chamado de tradução de endereços
- Endereços virtuais (lógicos)
 - Local usado pelo programa para acessar dados
 - Não existe do ponto de vista físico, mas é usado como base para definir o endereço físico (mapeamento)
 - São traduzidos para endereços físicos pela MMU, dispositivo denominado de Unidade de Gerenciamento de Memória
 - É manipulado em espaço de usuário (user space)

ENDEREÇOS FÍSICOS E VIRTUAIS

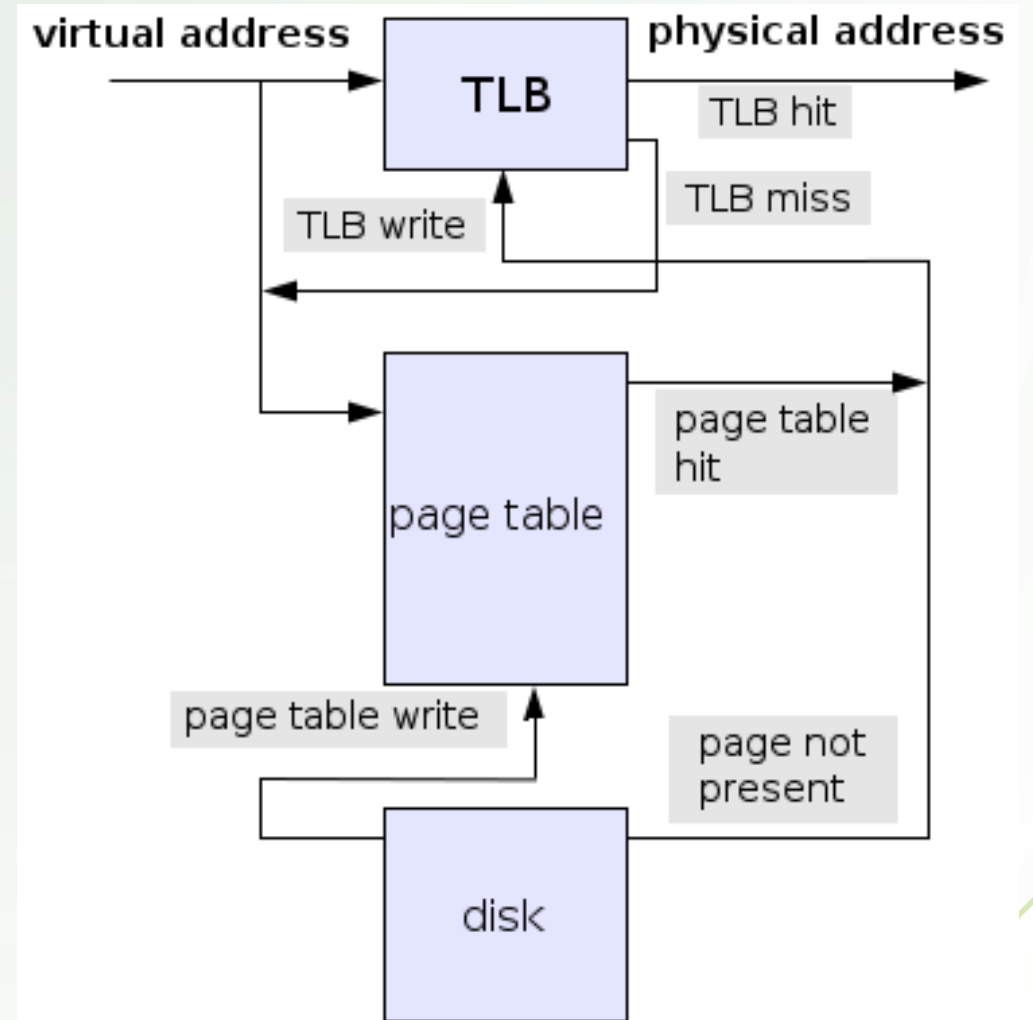
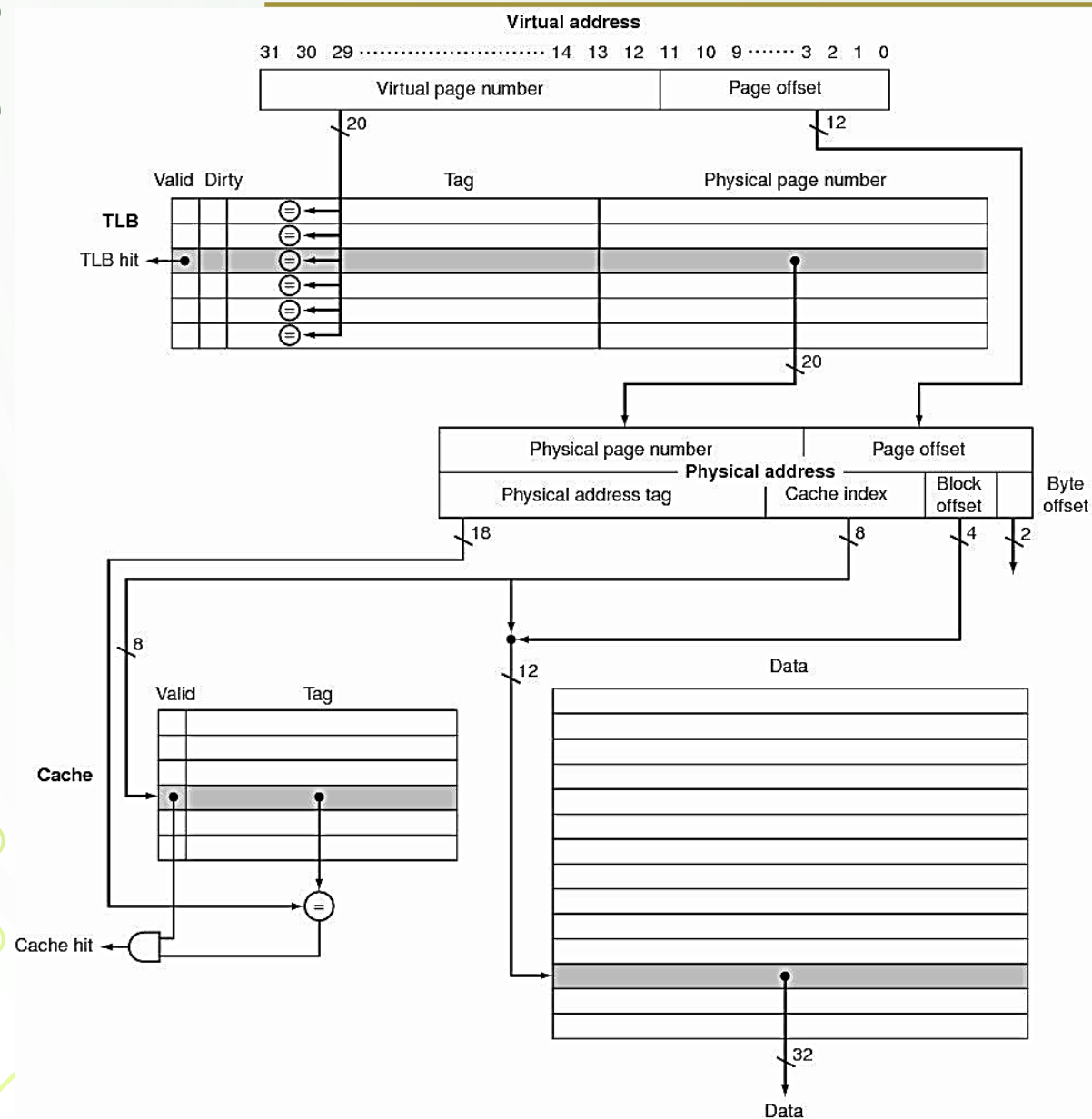


¹ Fonte: https://en.wikipedia.org/wiki/Physical_address

- TLB (Translation lookaside buffer):

- Memória Cache que armazena endereços de blocos de memória (memory pages)
- Circuito integrante da unidade de gerenciamento de memória (MMU)
- Fica posicionada entre o processador e a memória Ram
- Funciona como um acelerador (Cache) para verificar se um bloco está na Cache ou na Ram
- Portanto, usada para verificar se o bloco está na Cache (hit) ou na Ram (miss)

TLB-TRANSLATION LOOKASIDE BUFFER



MEMÓRIA DE TROCA (SWAP)

- Permite “simular” uma memória RAM com mais espaço que a memória física
- Utiliza o disco como meio de armazenamento
- Podendo ser armazenada em um arquivo ou em uma partição dedicada
- Dependendo da demanda de memória pelos programas
 - Maior pode ser o uso da memória Swap
 - Mais lento será o acesso à memória
- É operada através de endereços virtuais
 - Possuindo cada programa seu próprio espaço de endereçamento virtual
 - Utilizados para realizar operações sobre a memória

-
- Processos (programas em execução) não manipulam dados armazenados em disco (memória secundária) diretamente
 - Ao acessar uma área da memória de troca os dados precisam ser transferidos para a memória principal
 - Caso não haja espaço, precisa ocorrer uma troca: swap ↔ principal
 - Processos utilizam endereços virtuais
 - A memória principal utiliza endereços físicos
 - Assim, durante a troca é necessário realiza a tradução de endereços
 - Mecanismo de tradução do endereço virtual para endereço físico correspondente

- **Tabela de páginas**

- Implementa uma estrutura de dados em memória
- Usada pelo sistema de memória virtual do sistema operacional
- Armazena o **mapeamento** de endereços virtuais para físicos
- Usado para verificar se uma determinada página a ser acessada está ou não armazenada na memória física
- Componente importante do mecanismo de tradução de endereços virtuais

• Mecanismo de Paginação

- O espaço de **endereço virtual** de um processo é dividido em **páginas de tamanho fixo** (0,5 KB a 64 KB)
- O espaço de **endereçamento físico** é dividido em áreas proporcionais ao **tamanho da página**
- As páginas somente são levadas para a memória principal quando requisitadas por um processo
- Programas são desenvolvidos considerando memória principal suficiente para seu espaço de endereço virtual, mesmo não existindo
- **MMU** (Memory Management Unit):
 - Compreende a unidade de gerenciamento de memória
 - Responsável pelo **mapeamento de endereços virtual para físico**

Cenário exemplo

- **Memória virtual**

- Espaço de endereço virtual: 64 KB ($16 \times 4K$) (iniciais)
- Tamanho da página: 4 KB
- Total de páginas: 16 ($64/4$)
- Endereço virtual: 32 bits

- **Memória física**

- Espaço de endereço físico: 32 KB ($8 \times 4K$)
- Tamanho do frame (quadro): 4 KB
- Total de frames: 8 ($32/4$)
- Endereço físico: 15 bits

- **Tabela de páginas**

- Memória virtual: 16 entradas
- Memória física: 8 entradas

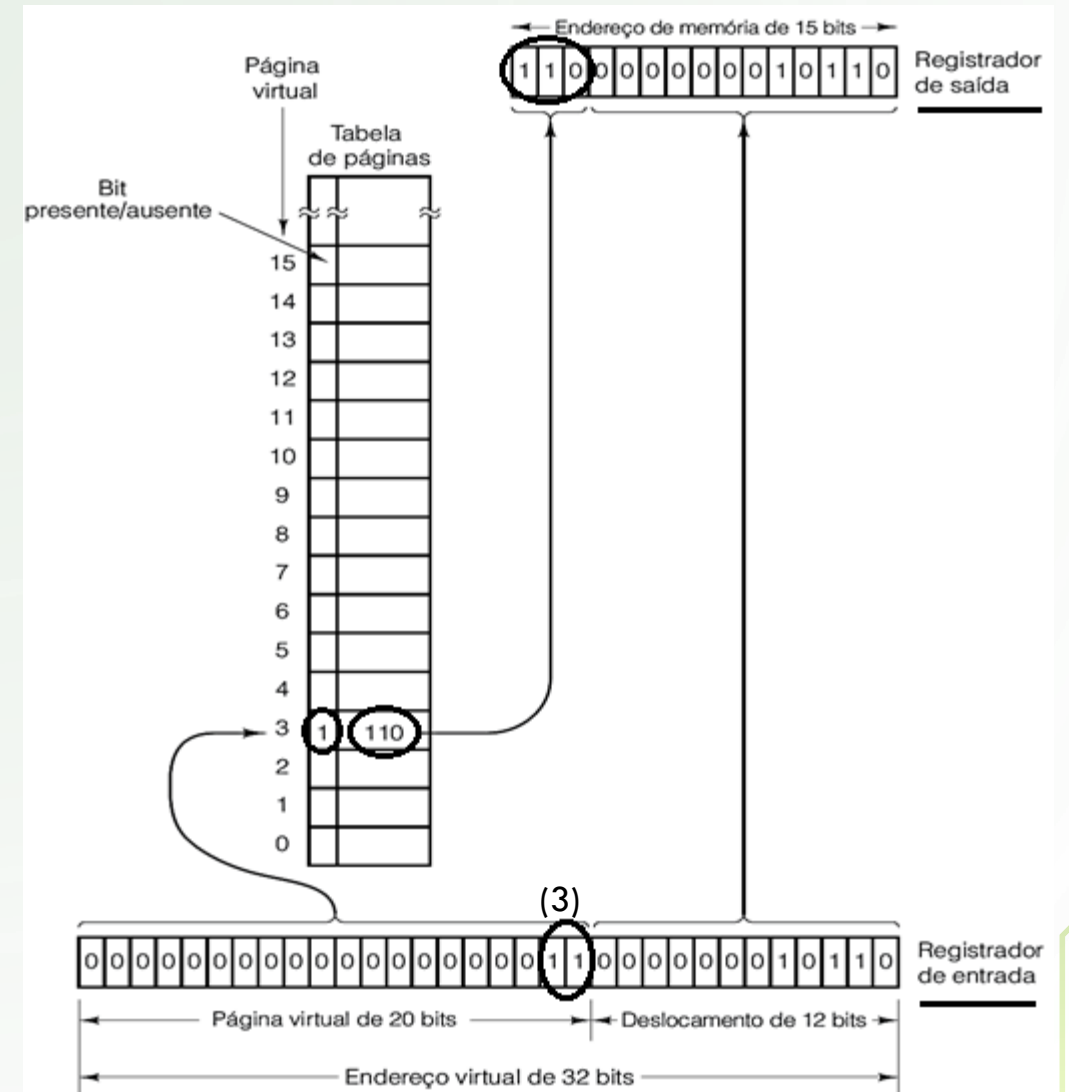
Quadro de página Endereços virtuais	
15	61440 – 65535
14	57344 – 61439
13	53248 – 57343
12	49152 – 53247
11	45056 – 49151
10	40960 – 45055
9	36864 – 40959
8	32768 – 36863
7	28672 – 32767
6	24576 – 28671
5	20480 – 24575
4	16384 – 20479
3	12288 – 16383
2	8192 – 12287
1	4096 – 8191
0	0 – 4095

32 KB da parte inferior da memória principal	
Quadro de página	Endereços físicos
7	28672 – 32767
6	24576 – 28671
5	20480 – 24575
4	16384 – 20479
3	12288 – 16383
2	8192 – 12287
1	4096 – 8191
0	0 – 4095

MMU: memory management unit

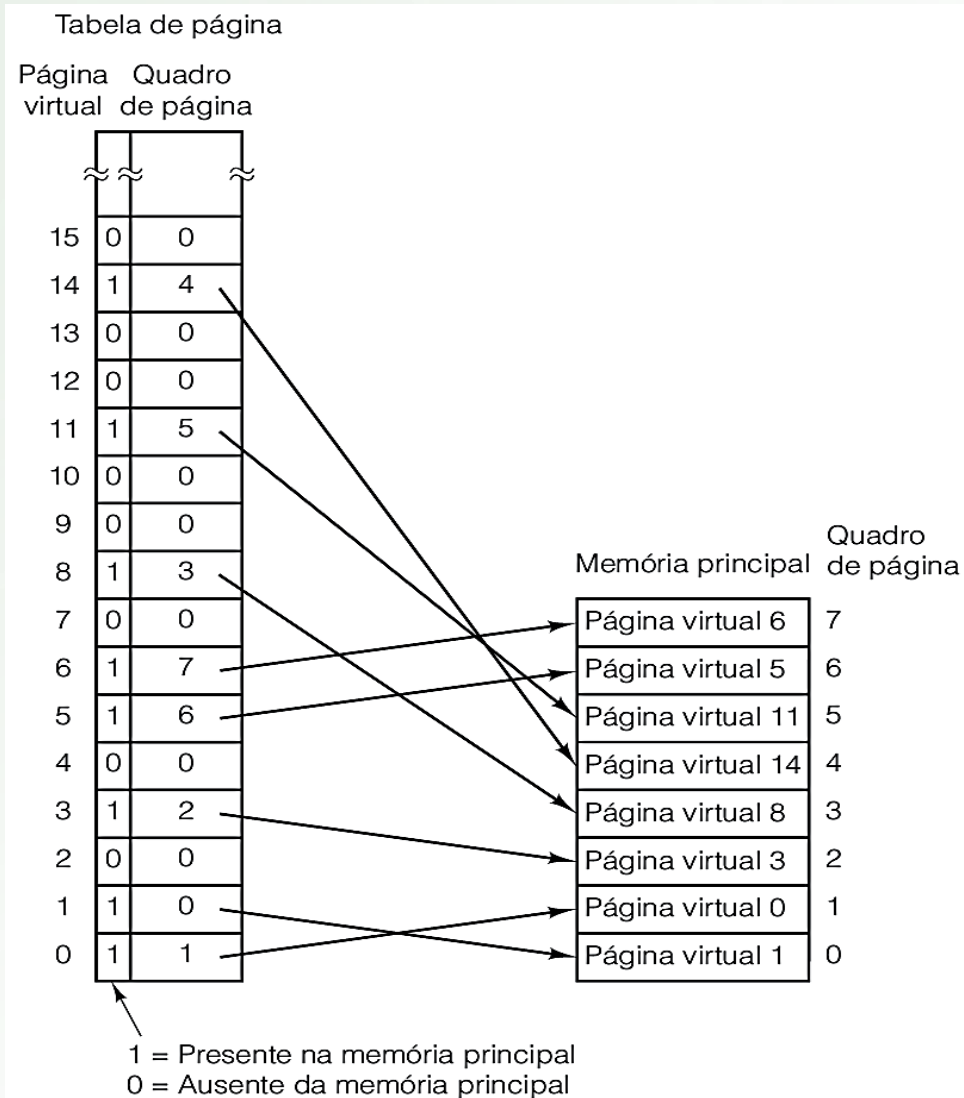
- Mapeamento de endereçamento:
virtual → físico

- Endereço virtual → 32 bits (entrada)
 - Endereço da página virtual → 20 bits
Usado como índice na tabela de páginas
 - Endereço de offset na página virtual → 12 bits
Relativo ao tamanho da página → 4K
 - Bit de presença: indica se a página está na memória física (1)
 - Endereço físico → 15 bits (saída)
-
- Endereço do frame físico → 3 bits
Obtido da entrada na tabela de página
 - Endereço de offset na página virtual → 12 bits
Relativo ao tamanho da página → 4K



Exemplo de mapeamento virtual → físico

- Mapeia as 8 páginas (virtuais, de 16) para os 8 quadros (físicos)
- Entradas da tabela com bit de presença:
 - ativo (1) estão em memória
 - inativo (0) estão em disco



- **Paginação por demanda**

- Páginas de memória são transferidas para a memória principal quando solicitadas pelo processo
- As páginas são buscadas (fetch) quando **referenciadas**
 - Apenas as páginas necessárias ficam na memória principal
 - Otimiza o tempo perdido com transferências
 - Pode ocorrer falha de página: página ausente na memória principal
- Política de substituição de página
 - Quando processo referencia uma página ausente da memória principal, ocorre a busca da página requisitada
 - Exige maior tempo, devido à transferência da memória secundária

- Política de substituição de página

- Ocorre quando um processo referencia uma página ausente e precisa inserir em uma entrada já ocupada
- Pode ser necessário escolher uma página da memória principal a ser substituída
- Para isto, precisamos de mecanismos de substituição
- Algoritmo LRU (Least Recently Used):
escolhe a página menos recentemente usada
- Algoritmo FIFO (First-In First-Out):
usa o mecanismo “primeiro a entrar, primeiro a sair”

- Fragmentação interna

- Dependendo do tamanho do programa pode existir uma última página do quadro **não complementando usada** (preenchida)

- Exemplo de cenário:

- Tamanho do programa: 26.000 bytes
 - Dividindo em páginas de 4.096 bytes (4kB), temos:
 - 6 páginas de 4.096 bytes (24576 B)
 - 1 página com 1.424 bytes desperdiçando 2.672^* bytes
 - * $4096 - 1424$
- Tamanho de páginas maiores aumentam a fragmentação, porém melhoram o desempenho
- Tamanho de páginas menores diminuem a fragmentação, porém diminuem o desempenho

- Segmentação

- Técnica que divide a **em segmentos** área de endereçamento de um programa
- Cada segmento constitui um **espaço de endereço separado**
- Diferentes segmentos podem ser expandidos ou encolhidos independentemente
- Todo a área do segmento é movimentado do disco para a memória principal (**swapping**)
- Segmentos não precisam ter o mesmo tamanho

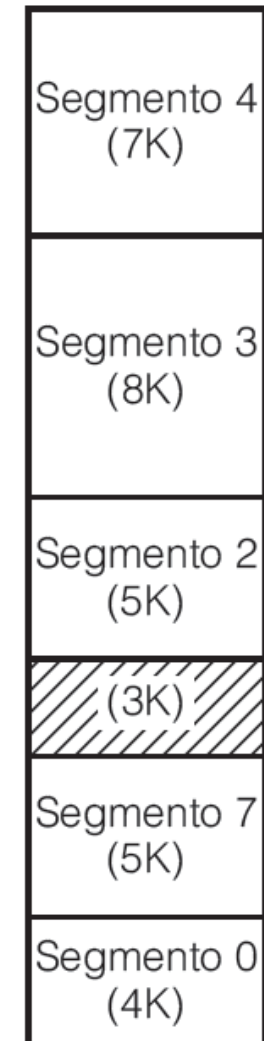
- **Situação 1:**

- A memória principal é preenchida com segmentos de vários tamanhos:
como os seguimento 0 a 4 ao lado



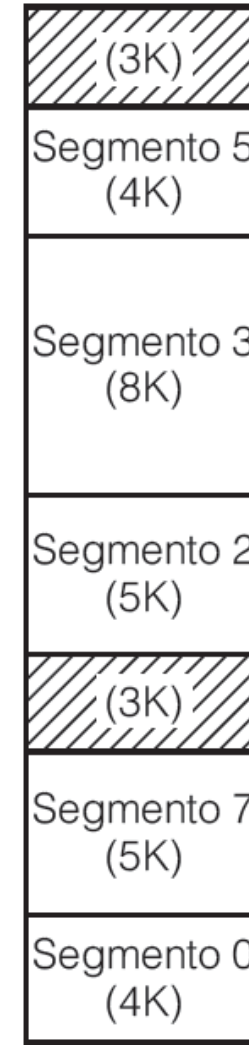
- **Situação 2:**

- O segmento 1 finaliza sua operação
- O segmento 7 ocupa sua posição

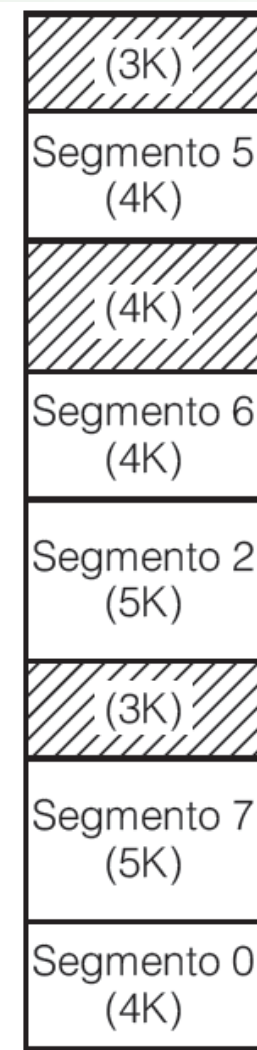


- **Situação 3:**

- O segmento 4 finaliza sua operação
- O segmento 5 ocupa sua posição



- Situação 4:
 - Após diversos segmentos finalizarem sua operação e diversas substituição de novos segmentos acontecerem, ocorre:
o fenômeno da **fragmentação** de memória
- As situações mencionadas causam a fragmentação externa
- Problema com a fragmentação externa:
 - Áreas de memória livres tornam-se inutilizadas



- Solução para fragmentação externa:

- Realizar uma compactação de memória
- Os segmentos são realocados de forma contínua
- O espaço restante torna-se acessível para outros segmentos
- Pode ser realizado entre regiões de memória principal
- Pode também ser realizado entre memória principal e secundária (disco), técnica conhecida como permutação (swapping)

- Problema:

- Tempo consumido pela reorganização e cópias



- Outra solução para fragmentação externa

- Uso de paginação
- Divide os segmentos em blocos de tamanhos fixos (páginas) acessadas sob demanda
- Partes de segmentos podem estar na memória principal ou em disco
- Requer o uso de tabela de páginas separadas por segmento

Paginação x Segmentação: quadro comparativo

Consideração	Paginação	Segmentação
O programador precisa estar ciente dela?	Não	Sim
Quantos espaços de endereços lineares há?	1	Muitos
O espaço de endereço virtual pode ser maior do que o tamanho da memória?	Sim	Sim
Tabelas de tamanhos variáveis podem ser manipuladas com facilidade?	Não	Sim
Por que a técnica foi inventada?	Para simular memórias grandes	Para fornecer vários espaços de endereço